

ONEDRAFT

CAD-AI — Aria Powered

Geometry & Parametric Model Engine

Version 0.1

Status: Implementation Specification

Database: PostgreSQL

Geometry Engine: Parametric / Computational Geometry Service

API: REST / JSON through `/api/v1`

Identifiers: UUID

Primary Model: Versioned OneDraft Project Model

AI Layer: Aria Powered

Coordinate Model: Project-local engineering coordinate system

Units: Metric-first, unit-aware

Primary Output: Deterministic architectural geometry and parametric building model

1. ENGINEERING PURPOSE

The Geometry & Parametric Model Engine is the computational layer between the OneDraft Project Model and the documentation system.

It is responsible for converting semantic architectural objects into deterministic geometric representations.

The engine must support:

buildings

levels

grids

spaces

walls

doors

windows

slabs

roofs

columns

beams

foundations

stairs

ramps

openings

assemblies

connections

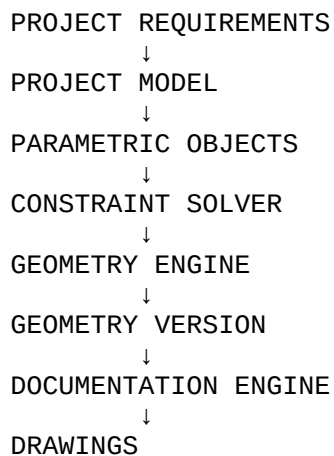
annotations references

and future architectural, structural, mechanical, electrical and civil objects.

The geometry engine does not replace the Project Model.

It derives geometry from the Project Model.

The fundamental relationship is:



2. CORE ARCHITECTURAL PRINCIPLE

OneDraft must never treat a drawing as the authoritative representation of the building.

The authoritative representation is:

SEMANTIC PROJECT MODEL
+
PARAMETERS
+
RELATIONSHIPS
+
CONSTRAINTS

+
GEOMETRY

A drawing is generated from that state.

Therefore:

MODEL → DRAWING

rather than:

DRAWING → MODEL

This permits the same project state to generate:

plans

elevations

sections

details

3D views

schedules

quantities

coordination information

rendering geometry

and future export formats.

3. GEOMETRY ENGINE RESPONSIBILITIES

The geometry engine shall provide:

1. Coordinate transformation
2. Parametric object generation
3. Geometric relationships
4. Constraint evaluation
5. Dependency propagation
6. Boolean operations
7. Intersection detection
8. Boundary generation

9. Solid generation
 10. Surface generation
 11. Opening generation
 12. Join and connection logic
 13. Spatial containment
 14. Collision detection
 15. Geometry validation
 16. Geometry hashing
 17. Geometry versioning
 18. Incremental regeneration
 19. Derived geometry generation
 20. Export geometry
-

4. GEOMETRY SERVICE BOUNDARY

The geometry engine shall operate as an internal service.

The external architecture remains:

```
CLIENT
↓
API
↓
DOMAIN SERVICE
↓
GEOMETRY SERVICE
↓
GEOMETRY WORKER
↓
GEOMETRY STORAGE
```

The client must not directly manipulate the geometry database.

Aria must not directly manipulate geometry storage.

Aria generates commands.

The Project Model service validates and applies commands.

The geometry engine generates geometry from the resulting model state.

5. GEOMETRY OBJECT MODEL

Every geometric object shall contain a common computational identity.

Conceptually:

GeometryObject

```
id
object_id
project_id
model_version_id
geometry_version
geometry_type
coordinate_system
parameters
dependencies
geometry_hash
bounding_box
status
```

The `object_id` identifies the semantic model object.

The geometry identifier identifies the computational representation.

These must remain distinct.

6. SEMANTIC OBJECT VERSUS GEOMETRIC OBJECT

For example:

```
model.elements.id
```

may identify:

```
Wall W-101
```

while:

```
geometry.geometry_references.id
```

identifies:

```
the geometric representation of Wall W-101
```

The semantic object contains meaning.

The geometry object contains computational form.

7. GEOMETRY TYPES

The initial geometry system shall support:

POINT
VECTOR
LINE
POLYLINE
ARC
CIRCLE
ELLIPSE
CURVE
POLYGON
SURFACE
MESH
SOLID
BREP
INSTANCE
TRANSFORM

The MVP should prioritize:

POINT
LINE
POLYLINE
ARC
POLYGON
SURFACE
SOLID

8. COORDINATE SYSTEM

Every project shall possess a project-local coordinate system.

The base coordinate system is:

X = East / West
Y = North / South
Z = Elevation

The origin shall be project-defined.

Conceptually:

Origin
(0, 0, 0)

X →
Y ↑
Z ↑

The project coordinate system must remain independent from screen coordinates.

9. SURVEY COORDINATE SYSTEM

A project may contain an external survey coordinate system.

The engine shall maintain a transformation:

SURVEY COORDINATES

↓

PROJECT COORDINATES

The transformation shall include:

translation
rotation
scale
elevation offset

The transformation itself must be versioned.

10. UNITS

The internal geometry engine shall use a consistent base unit.

The MVP base unit shall be:

millimetres

Angles shall use:

degrees

Internally, computational routines may convert angles to radians where required.

All API quantities must remain explicitly unit-aware.

Example:

```
{  
  "value": 300,  
  "unit": "mm"  
}
```

11. PARAMETRIC OBJECT

A parametric object is defined by parameters rather than only by static geometry.

Example wall:

Wall

```
start_point
end_point
height
thickness
base_elevation
top_elevation
assembly
openings
justification
```

Geometry is generated from those parameters.

12. PARAMETRIC WALL

Initial wall definition:

WallParameters

```
start
end
height
thickness
base_elevation
top_elevation
justification
assembly_id
location_line
```

The wall generator produces:

```
centerline
reference faces
side faces
top face
bottom face
solid
bounding box
```

13. WALL JUSTIFICATION

Wall placement shall support:

```
CENTER
CORE_CENTER
INTERIOR_FACE
EXTERIOR_FACE
LEFT
RIGHT
```

The geometric offset shall be calculated from the assembly thickness and justification rule.

14. WALL ASSEMBLY

A wall shall reference an assembly.

Example:

Assembly
Exterior Finish
Sheathing
Stud Cavity
Vapour Control
Interior Finish

The geometry engine may generate either:

single composite solid

or:

layered solids

depending upon the requested level of geometric detail.

15. LEVEL DATUM

Each level establishes a vertical reference.

Example:

P2
P1
G
L1
L2
L3
ROOF

Each level contains:

elevation
floor thickness
floor finish
ceiling elevation
sequence

The geometry engine uses these values to establish vertical placement.

16. LEVEL DEPENDENCY

An element assigned to a level inherits a default vertical coordinate system.

Example:

```
Level L1  
Elevation = 3200 mm
```

A wall with:

```
base_offset = 0  
height = 3000
```

generates:

```
base Z = 3200  
top Z = 6200
```

unless another constraint overrides the value.

17. BUILDING GRID

Grids are first-class parametric references.

A grid may be:

```
linear  
radial  
orthogonal  
custom
```

Example:

```
A  
B  
C  
D
```

```
1  
2  
3  
4
```

Elements may reference grids rather than storing only absolute coordinates.

18. GRID-BASED PLACEMENT

An object may define:

grid_reference
offset
alignment

Example:

Wall:
Grid A → Grid C
Offset = 150 mm

The geometry engine resolves the grid relationship into actual coordinates.

19. SPACES

Spaces shall be derived from boundaries.

A space is not merely a manually entered area.

Its area is calculated from its boundary geometry.

Conceptually:

BOUNDARY
↓
CLOSED LOOP
↓
PLANAR REGION
↓
AREA
↓
VOLUME

20. SPACE BOUNDARY

The engine shall detect or explicitly receive:

boundary curves

The curves must form a valid closed region.

Validation must identify:

open boundary
self-intersection
duplicate edge
zero-length edge
invalid orientation

21. SPACE AREA

For a valid planar region:

Area = geometric area of region

The calculated value shall be stored as derived data.

It must be possible to regenerate it from the geometry.

22. SPACE VOLUME

Where a valid vertical extent exists:

Volume = Area × Height

For irregular spaces the engine shall use solid intersection and volume calculation.

23. DOORS

A door is a parametric opening and architectural object.

Initial parameters:

width
height
sill_elevation
head_elevation
host_wall
swing_direction
swing_angle
frame_depth
type

The door geometry consists of:

opening
frame
leaf
hardware reference
swing representation

24. WINDOWS

Window parameters:

width
height
sill_height
head_height
host_wall
frame_depth
operation
glazing_type

The window generator creates the opening and associated geometry.

25. OPENING ENGINE

Openings must be independent computational objects.

The opening engine shall support:

door openings
window openings
mechanical openings
structural openings
service penetrations
custom openings

An opening references its host object.

26. BOOLEAN OPERATIONS

The geometry engine shall support:

UNION
SUBTRACT
INTERSECT
SPLIT
TRIM
EXTEND
OFFSET
FILLET
CHAMFER

MVP priority:

UNION
SUBTRACT
INTERSECT
SPLIT
OFFSET

27. HOST / VOID RELATIONSHIP

Example:

Wall
↓
Window
↓
Opening

The wall remains the semantic host.

The opening is a derived geometric subtraction.

If the window moves:

window parameters change
↓
opening changes
↓
wall geometry regenerates
↓
affected drawings regenerate

28. DEPENDENCY GRAPH

Geometry generation must maintain a dependency graph.

Example:

Level
↓
Wall
↓
Window
↓
Opening
↓
Wall Solid
↓
Space Boundary
↓
Space Area
↓
Plan View
↓
Drawing

This allows incremental regeneration.

29. DEPENDENCY RECORD

Dependencies shall conceptually contain:

```
source_object_id
target_object_id
dependency_type
parameters
created_at
```

Existing:

```
model.relationships
```

shall remain the authoritative semantic relationship graph.

The geometry service may maintain an optimized execution graph.

30. CONSTRAINT ENGINE

The geometry engine must not blindly execute parameters.

Before committing geometry, it evaluates active constraints.

Constraint classes include:

```
DIMENSIONAL
ALIGNMENT
DISTANCE
ANGLE
PARALLEL
PERPENDICULAR
COINCIDENT
CONTAINMENT
CLEARANCE
BOUNDARY
LEVEL
HOST
DEPENDENCY
CODE-DERIVED
```

31. HARD CONSTRAINTS

Hard constraints cannot be violated by an ordinary command.

Examples:

```
element must remain inside property boundary
opening must remain inside host wall
level elevation must remain valid
```

required clearance must remain satisfied

If a hard constraint fails:

COMMAND → REJECTED

No model revision is created.

32. SOFT CONSTRAINTS

Soft constraints may be violated only when explicitly permitted.

Example:

preferred room dimension
preferred alignment
preferred grid spacing

The system records the deviation.

33. CONSTRAINT CONFLICT

A conflict shall produce a structured result:

```
{  
  "status": "CONFLICT",  
  "constraint_id": "uuid",  
  "object_id": "uuid",  
  "severity": "ERROR",  
  "message": "Requested movement violates required clearance.",  
  "resolution_options": []  
}
```

The geometry engine must not silently resolve a conflict by changing unrelated project geometry.

34. PARAMETRIC SOLVING

The initial solver shall support deterministic direct solving.

Example:

Wall A
↓
3000 mm

Wall B
↓

5000 mm

Room width

↓

Wall B - Wall A

Future versions may introduce a generalized constraint solver.

35. SOLVER PRIORITY

When multiple constraints affect an object, priority shall be:

1. Legal / code constraints
2. Project hard constraints
3. Structural constraints
4. Safety constraints
5. Explicit user constraints
6. Model relationships
7. Design preferences
8. Default geometry behavior

A lower-priority rule must not silently override a higher-priority rule.

36. ARIA GEOMETRY COMMAND

Aria shall communicate geometry changes through domain commands.

Example:

```
{
  "operation": "MOVE_ELEMENT",
  "target_objects": [
    "uuid"
  ],
  "parameters": {
    "distance": 300,
    "unit": "mm",
    "direction": "NORTH"
  }
}
```

The geometry engine does not accept arbitrary natural-language instructions.

The Project Model service translates and validates the command first.

37. GEOMETRY COMMAND PIPELINE

ARIA
↓
COMMAND
↓
AUTHORIZATION
↓
PROJECT MODEL VALIDATION
↓
CONSTRAINT EVALUATION
↓
PARAMETER UPDATE
↓
GEOMETRY REBUILD
↓
GEOMETRY VALIDATION
↓
REVISION
↓
DOCUMENTATION INVALIDATION

38. GEOMETRY BUILD REQUEST

Internal request:

```
{  
  "project_id": "uuid",  
  "model_version_id": "uuid",  
  "objects": [  
    "uuid",  
    "uuid"  
  ],  
  "build_mode": "INCREMENTAL"  
}
```

Build modes:

FULL
INCREMENTAL
OBJECT
VIEW
EXPORT

39. GEOMETRY BUILD RESPONSE

```
{  
  "project_id": "uuid",  
  "model_version_id": "uuid",  
  "status": "COMPLETED",  
}
```

```
"geometry_version": 17,  
"objects_processed": 42,  
"objects_changed": 7,  
"geometry_hash": "hash"  
}
```

40. FULL REBUILD

A full rebuild reconstructs geometry from the authoritative model state.

```
PROJECT MODEL  
↓  
PARAMETERS  
↓  
DEPENDENCIES  
↓  
CONSTRAINTS  
↓  
GEOMETRY  
↓  
VALIDATION  
↓  
HASH
```

A full rebuild is the definitive reconstruction mechanism.

41. INCREMENTAL REBUILD

An incremental rebuild processes only affected objects and their downstream dependencies.

Example:

```
Move Wall  
↓  
Door  
↓  
Window  
↓  
Space  
↓  
Plan  
↓  
Section
```

Unrelated building elements remain unchanged.

42. GEOMETRY INVALIDATION

When a model object changes, dependent geometry is marked:

CURRENT
STALE
INVALID
REBUILDING
VALID

Stale geometry must never be presented as current.

43. GEOMETRY VERSIONING

Each generated geometry state receives:

geometry_version

The geometry record must reference:

project_id
model_version_id
object_id
geometry_version

This makes geometry reproducible.

44. GEOMETRY HASH

Every geometry payload shall produce a deterministic hash.

Conceptually:

```
HASH(  
    geometry_type  
    coordinates  
    topology  
    units  
    coordinate_system  
    parameters  
)
```

The hash is stored with the geometry reference.

45. DETERMINISM

Given identical:

Project Model
Model Version
Parameters
Constraints
Geometry Engine Version

the engine must produce the same geometry.

This is a core architectural requirement.

46. ENGINE VERSION

Every geometry build shall record the engine version.

Example:

```
geometry_engine_version = 0.1.0
```

This permits historical geometry to be distinguished from geometry generated by later algorithms.

47. REPRODUCTION RECORD

A geometry build shall preserve:

```
project_id  
model_version_id  
geometry_engine_version  
input_hash  
output_hash  
parameter_hash  
constraint_hash  
created_at
```

This establishes computational provenance.

48. BOUNDING BOX

Every geometric object shall have an axis-aligned bounding box.

```
{  
  "min": {  
    "x": 0,
```

```
    "y": 0,  
    "z": 0  
  },  
  "max": {  
    "x": 5000,  
    "y": 3000,  
    "z": 3000  
  }  
}
```

Bounding boxes are used for:

- collision filtering
- spatial queries
- view generation
- section clipping
- performance optimization

49. SPATIAL INDEX

The geometry service shall maintain an optimized spatial index.

The initial implementation may use:

R-tree

or the equivalent spatial indexing capability of the selected geometry library.

The index is derived.

It is not the authoritative model state.

50. COLLISION DETECTION

The engine shall support collision queries between:

- architectural elements
- structural elements
- openings
- assemblies
- equipment
- service penetrations

Collision results are validation outputs.

They do not automatically modify the model.

51. CLEARANCE DETECTION

The engine shall support minimum-distance queries.

Example:

Door
↓
Required clearance
↓
Adjacent wall

Result:

PASS

or:

FAIL

52. INTERSECTION ENGINE

The engine shall identify:

line-line intersections
curve-curve intersections
curve-surface intersections
solid-solid intersections
solid-plane intersections

These operations become foundational for:

sections
elevations
openings
joins
coordination
documentation

53. SECTION GENERATION

A section is generated by intersecting the model with a section plane.

3D MODEL
↓
SECTION PLANE
↓
INTERSECTION
↓

CUT GEOMETRY
↓
PROJECTED GEOMETRY
↓
SECTION VIEW

54. ELEVATION GENERATION

An elevation is generated by projecting relevant geometry onto an elevation plane.

The engine determines:

visible geometry
occluding geometry
silhouette
openings
major edges
levels
grids

The documentation engine then applies graphic standards.

55. PLAN GENERATION

A plan view is generated from a horizontal cut plane.

MODEL
↓
CUT ELEVATION
↓
CUT ELEMENTS
↓
BELOW-CUT ELEMENTS
↓
ABOVE-CUT ELEMENTS
↓
VIEW GRAPH

Graphic representation remains a documentation concern.

56. VIEW DEPENDENCY

A view references:

model_version_id
view_definition
cut_plane

projection_direction
visibility_settings
scale

Therefore a view is reproducible.

57. GEOMETRY TO DOCUMENTATION

The geometry engine produces model geometry.

The documentation engine converts geometry into:

view geometry
linework
filled regions
symbol references
dimensions
annotations
schedules

The geometry engine must not embed sheet-specific graphic rules into the core model.

58. MODEL SPACE VERSUS PAPER SPACE

The geometry engine operates in:

MODEL SPACE

The documentation engine operates in:

VIEW SPACE
PAPER SPACE

The distinction must remain explicit.

59. TRANSFORM SYSTEM

Every view shall support transformations:

translation
rotation
scale
projection
clipping

The transformation chain must be deterministic.

60. OBJECT LOCAL COORDINATES

Complex parametric objects may define local coordinate systems.

Example:

```
Door Local Coordinates
  X = width
  Y = depth
  Z = height
```

The object is then transformed into project coordinates.

This allows reusable component definitions.

61. COMPONENT INSTANCING

Reusable objects may be represented as instances.

Example:

```
Door Type D01
  ↓
Instance 001
Instance 002
Instance 003
```

The type defines shared parameters.

The instance defines placement and instance-specific overrides.

62. FAMILY / TYPE SYSTEM

The MVP shall support a basic type system:

```
Element Type
  ↓
Type Parameters
  ↓
Instance Parameters
  ↓
Geometry
```

Examples:

```
WALL-EXT-01
DOOR-900
```

63. TYPE PARAMETER OVERRIDE

An instance may override permitted parameters.

Example:

Window Type:
width = 1200
height = 1500

Instance:
width = 900
height = 1500

Parameters not marked overridable remain controlled by the type.

64. ASSEMBLY GEOMETRY

Assemblies shall be capable of generating:

overall solid
individual layers
material boundaries
layer thickness
finish faces

The required level of detail depends upon the requested output.

65. DETAIL LEVEL

The engine shall support:

LOD_100
LOD_200
LOD_300
LOD_400

The initial MVP may implement:

SCHEMATIC
DOCUMENTATION
FABRICATION

as simplified internal detail levels.

66. GEOMETRY TOLERANCE

Computational geometry requires explicit tolerance.

The engine shall define:

```
linear_tolerance  
angular_tolerance  
coincidence_tolerance  
intersection_tolerance
```

Example:

```
linear_tolerance = 0.01 mm
```

The actual production value shall be established through engine testing.

67. NUMERICAL ROBUSTNESS

The engine must avoid:

```
floating-point equality comparisons
```

where tolerance-based comparisons are required.

Conceptually:

```
abs(a - b) <= tolerance
```

is used for approximate equality.

68. GEOMETRY VALIDATION

Every completed build shall evaluate:

```
valid topology  
closed solids  
non-zero dimensions  
non-self-intersecting boundaries  
valid coordinates  
valid normals  
valid transforms  
valid host relationships  
valid openings
```

69. INVALID GEOMETRY

Invalid geometry receives:

```
status = INVALID
```

The engine shall return:

```
object_id  
geometry_version  
failure_type  
failure_message  
diagnostic_data
```

Invalid geometry must not become the active geometry version.

70. TOPOLOGY

Solid geometry shall maintain:

```
vertices  
edges  
faces  
shells  
solids
```

Topology must remain internally consistent.

71. FACE ORIENTATION

Closed solids require consistent face orientation.

The geometry validator shall detect:

```
inverted faces  
duplicate faces  
open shells  
non-manifold edges
```

72. MANIFOLD REQUIREMENT

Where a solid is expected, the geometry engine should require a valid manifold representation.

Non-manifold geometry may be permitted only for specific intermediate computational operations.

73. PARAMETER VALIDATION

Before geometry generation, parameters are validated.

Example:

```
wall.height > 0  
wall.thickness > 0  
door.width > 0  
door.height > 0
```

Invalid parameters produce:

PARAMETER_ERROR

before expensive geometry computation begins.

74. TRANSACTION INTEGRATION

The geometry engine does not independently commit model changes.

The Project Model service controls the transaction.

The architecture is:

```
COMMAND  
↓  
BEGIN TRANSACTION  
↓  
UPDATE MODEL  
↓  
COMMIT MODEL VERSION  
↓  
BUILD GEOMETRY  
↓  
VALIDATE GEOMETRY  
↓  
ACTIVATE GEOMETRY
```

If geometry is required to succeed before finalization, the domain service may use a staged model state.

75. STAGED MODEL STATE

For complex operations:

CURRENT MODEL
↓
PROPOSED MODEL
↓
GEOMETRY BUILD
↓
VALIDATION
↓
ACCEPT

Only the accepted state becomes current.

76. ARIA PROPOSAL WORKFLOW

Aria may propose:

change parameters
add object
remove object
move object
resize object
change assembly
change relationship
change constraint

The proposal is evaluated before becoming a committed revision.

77. PROPOSAL EVALUATION

ARIA PROPOSAL
↓
PARAMETER VALIDATION
↓
CONSTRAINT CHECK
↓
GEOMETRY PREVIEW
↓
GEOMETRY VALIDATION
↓
DOCUMENT IMPACT ANALYSIS
↓
PROPOSED REVISION

The resulting proposal can then be accepted through the application workflow.

78. IMPACT ANALYSIS

Every geometry modification shall determine:

affected objects
affected spaces
affected views
affected drawings
affected schedules
affected validation results
affected document packages

This information is stored with the command result.

79. DOCUMENT INVALIDATION

If geometry changes after a drawing is generated, affected documentation receives:

STALE

status.

Example:

Wall changed
↓
Plan A101 → STALE
Section A301 → STALE
Elevation A201 → STALE

Unrelated documents remain current.

80. VALIDATION INVALIDATION

A model revision invalidates validation results tied to an earlier model version.

Therefore:

Revision 17
Validation 17

must never be represented as validation of:

Revision 18

unless explicitly rerun.

81. GEOMETRY JOB QUEUE

Expensive geometry operations shall execute asynchronously.

Example:

```
POST /internal/geometry/build
```

returns:

```
{
  "job_id": "uuid",
  "status": "QUEUED"
}
```

82. GEOMETRY WORKER

The geometry worker performs:

```
load model
load dependencies
load constraints
construct parameters
solve parameters
generate geometry
validate geometry
hash geometry
store geometry
emit completion event
```

83. WORKER ISOLATION

Geometry workers should operate independently of the API process.

This prevents computational geometry from blocking API request handling.

The architecture becomes:

```
API
↓
QUEUE
↓
GEOMETRY WORKER
↓
GEOMETRY STORE
```

84. JOB IDEMPOTENCY

Geometry jobs shall contain:

```
project_id
model_version_id
input_hash
build_mode
```

If an identical valid geometry result already exists, the worker may reuse it.

85. STALE JOB PROTECTION

A worker completing an old job must not overwrite newer geometry.

Before activation:

```
job.model_version_id
```

must match the intended target model version.

Otherwise:

JOB → STALE

86. GEOMETRY STORAGE

The relational database stores:

```
metadata
references
hashes
versions
relationships
```

Large geometry payloads may be stored in object storage.

Example:

```
PostgreSQL
  ↓
geometry_reference
  ↓
object storage
  ↓
geometry artifact
```

87. GEOMETRY ARTIFACT FORMAT

The engine shall support an internal canonical representation.

Possible representations include:

B-Rep
mesh
serialized computational geometry

The canonical format must be independent of a specific drawing export format.

88. EXPORT FORMATS

The geometry layer should eventually support:

DXF
DWG-compatible export pipeline
IFC
STEP
OBJ
glTF
STL
SVG linework
PDF vector linework

Export adapters remain outside the canonical geometry engine.

89. IFC INTEROPERABILITY

IFC export shall map semantic objects rather than simply exporting triangulated geometry.

Example:

OneDraft Wall
↓
IFC Wall

with properties retained where mapping is supported.

90. DXF / CAD LINEWORK

CAD export shall derive linework from:

model geometry
view definition

cut plane
projection
graphic settings

The engine must not permanently convert the project into 2D CAD linework.

91. 3D VIEW GENERATION

The engine shall provide renderable geometry for:

interactive 3D viewer
visual inspection
coordination
rendering

The 3D representation is derived from the same model version used for documentation.

92. MODEL QUERY ENGINE

The geometry subsystem shall support spatial queries:

objects within box
objects intersecting plane
objects near point
objects intersecting object
objects contained by space
objects hosted by element
objects visible from view

93. EXAMPLE SPATIAL QUERY

```
{  
  "query": "INTERSECTING",  
  "object_id": "uuid"  
}
```

Response:

```
{  
  "data": [  
    {  
      "object_id": "uuid",  
      "relationship": "INTERSECTS"  
    }  
  ]  
}
```

94. GEOMETRIC RELATIONSHIPS

The engine shall calculate:

COINCIDENT
TOUCHING
INTERSECTING
CONTAINING
CONTAINED_BY
PARALLEL
PERPENDICULAR
OFFSET
ADJACENT
CONNECTED

These may be stored as derived relationships.

95. MODEL RELATIONSHIP VERSUS GEOMETRIC RELATIONSHIP

A semantic relationship:

Door HOSTED_BY Wall

is different from a geometric relationship:

Door GEOMETRY_INTERSECTS Wall

Both may exist.

They must not be conflated.

96. FOUNDATION GEOMETRY

The initial building model shall support:

footings
foundation walls
slabs-on-grade
grade beams
pile references
foundation pads
retaining walls

Foundation geometry must reference building and level datums.

97. STRUCTURAL GEOMETRY

Initial structural objects:

columns
beams
joists
walls
slabs
braces
trusses
foundations

The architecture shall permit structural analysis integration later.

98. ROOF GEOMETRY

Initial roof parameters:

roof boundary
slope
ridge
valley
eave
overhang
pitch
thickness

The roof generator produces:

roof planes
intersections
edges
solid

99. STAIRS

Initial stair parameters:

base_level
top_level
width
riser_height
tread_depth
run
landing_length
direction

The engine calculates:

- riser count
- tread count
- total rise
- total run
- landing geometry

100. RAMPS

Ramp parameters:

- start_elevation
- end_elevation
- run
- width
- slope
- landing

The geometry engine shall calculate the resulting slope and verify constraints.

101. ACCESSIBILITY GEOMETRY

Accessibility constraints may be represented as geometric requirements.

Examples:

- clear door width
- turning radius
- clear floor area
- ramp slope
- landing dimensions
- circulation width

These are evaluated through the validation engine.

102. FIRE / LIFE-SAFETY GEOMETRY

The geometry layer shall expose spatial information required by compliance validation:

- travel distance
- exit access
- occupancy boundaries
- fire separations
- opening locations
- egress widths

The regulatory engine determines the applicable rule.

The geometry engine supplies the measurements.

103. CODE ENGINE SEPARATION

The geometry engine does not decide:

what the building code requires

It determines:

what the model geometrically contains

The compliance engine determines whether that geometry satisfies the applicable requirement.

104. QUANTITY COMPUTATION

The same geometry engine shall eventually support:

area
volume
length
count
surface area
material quantities

These quantities become inputs to schedules and estimating.

105. MATERIAL QUANTITIES

Example:

Wall Assembly
↓
Wall Area
↓
Layer Thickness
↓
Material Volume

The quantity engine must identify whether quantities are:

gross
net
deducted
waste-adjusted

106. GEOMETRY CACHE

Frequently reused geometry may be cached by:

input_hash
engine_version
geometry_type
detail_level

The cache is disposable.

It is never the system of record.

107. CACHE INVALIDATION

Cache invalidation occurs when:

parameters change
dependencies change
engine version changes
coordinate system changes
assembly changes
geometry tolerance changes

108. OBSERVABILITY

Every geometry job shall produce structured telemetry:

job_id
project_id
model_version_id
engine_version
start_time
end_time
duration
objects_processed
objects_failed
memory_usage
status

109. DIAGNOSTICS

Geometry failures must be actionable.

Example:

GEOMETRY_INVALID

Object: Wall W-102

Operation: BOOLEAN_SUBTRACT

Cause: Window opening produces non-manifold result

Input geometry hash: ...

Diagnostics shall identify the smallest useful failure boundary.

110. ERROR CLASSIFICATION

Initial geometry error classes:

PARAMETER_ERROR

COORDINATE_ERROR

TOPOLOGY_ERROR

BOOLEAN_ERROR

CONSTRAINT_ERROR

DEPENDENCY_ERROR

TRANSFORM_ERROR

NUMERICAL_ERROR

STORAGE_ERROR

STALE_MODEL_ERROR

111. TESTING STRATEGY

The geometry engine shall use:

unit tests

property tests

regression tests

integration tests

golden geometry tests

performance tests

112. GOLDEN GEOMETRY TESTS

A known project model is generated.

Its expected geometry hash is recorded.

Future engine builds are compared.

If the geometry changes unexpectedly:

REGRESSION

is reported.

113. PROPERTY TEST

The engine should verify properties such as:

wall length remains positive
closed polygons remain closed
generated solids have valid topology
moving an object preserves its dimensions
translation preserves geometry shape
rotation preserves distances

114. TRANSLATION INVARIANT

For a pure translation:

$\text{distance}(A, B)$

must remain unchanged.

Likewise:

area
volume
angles

remain invariant.

115. ROTATION INVARIANT

For a rigid rotation:

length
area
volume
angles

must remain invariant.

Only coordinates and orientation change.

116. PARAMETRIC REGRESSION

Example:

Wall length = 5000 mm
Wall height = 3000 mm
Wall thickness = 200 mm

Expected volume:

3,000,000,000 mm³

The test verifies the computational result.

117. OPENING REGRESSION

Example:

Wall:
5000 × 3000 × 200

Opening:
900 × 2100

The engine must generate a wall with a corresponding void.

The opening must remain associated with the host wall.

118. SECTION REGRESSION

Given an identical model and section plane:

section geometry hash

must remain deterministic.

119. PERFORMANCE TARGET

The geometry engine shall support incremental regeneration rather than rebuilding the entire building for every parameter change.

The initial performance objective is:

single-object modification
→ sub-second model evaluation where computationally practical

Large geometry operations may execute asynchronously.

120. LARGE PROJECT SCALING

The architecture must permit:

10,000+
100,000+
1,000,000+

geometric objects without requiring the entire model to be loaded into every worker operation.

Spatial indexing, dependency analysis and incremental computation become essential at scale.

121. PARTITIONING

Large projects may eventually partition geometry by:

building
level
discipline
zone
model region

The partition system must preserve a unified project coordinate system.

122. MULTI-BUILDING PROJECTS

A project may contain multiple buildings.

Example:

Project
├── Building A
├── Building B
└── Building C

Each building has independent local semantics while sharing the project coordinate system.

123. LINKED MODELS

Future versions may support linked models.

Example:

Architectural Model

↓

Structural Model

↓

Mechanical Model

Linked geometry remains externally owned but spatially queryable.

124. DISCIPLINE SEPARATION

The core geometry engine shall remain discipline-neutral.

Discipline-specific semantics belong to domain modules.

Examples:

architecture
structure
mechanical
electrical
plumbing
civil
landscape

125. MODEL OWNERSHIP

Each object must have a clear ownership boundary.

Conceptually:

project_id
discipline
source
authority

This prevents one subsystem from silently modifying another discipline's authoritative model.

126. ARIA AND GEOMETRY

Aria functions as the reasoning and orchestration layer.

The geometry engine functions as the deterministic computational layer.

Therefore:

ARIA:
"What should change?"

GEOMETRY ENGINE:
"What geometry results?"

VALIDATION ENGINE:
"Is the result acceptable?"

DOCUMENT ENGINE:
"How should it be documented?"

127. NO DIRECT AI GEOMETRY

Aria must not be treated as the geometry kernel.

AI output may contain:

intent
parameters
object references
constraints
operations

The geometry engine must independently compute the result.

128. COMMAND EXAMPLE

User intent:

Move the rear bedroom wall 300 mm north.

Aria resolves this into:

MOVE_ELEMENT
target = wall
distance = 300 mm
direction = north

The geometry engine then determines the resulting coordinates and dependent geometry.

129. COMMAND RESULT

The result must report:

command
model version

geometry version
affected objects
affected views
affected drawings
constraint results
validation status

130. PREVIEW MODE

The system should support:

PREVIEW

before:

COMMIT

Preview generates temporary geometry without replacing the current authoritative state.

131. PREVIEW ARCHITECTURE

CURRENT MODEL
↓
PROPOSED PARAMETERS
↓
TEMPORARY GEOMETRY
↓
VALIDATION
↓
USER REVIEW
↓
COMMIT

132. REDLINE INTEGRATION

A redline is interpreted as an instruction against the model.

Example:

REDLINE
↓
TARGET OBJECT
↓
ARIA INTERPRETATION
↓
COMMAND
↓

GEOMETRY
↓
VALIDATION
↓
RESOLUTION

A redline cannot be marked resolved merely because text was generated.

The model must actually change.

133. GEOMETRY ACCEPTANCE

A geometry result becomes active only when:

parameters valid
constraints satisfied
geometry valid
model version current

and, where required:

user approval

has been received.

134. FINAL GEOMETRY STATE

The active geometry state is:

VALID
CURRENT
HASHED
VERSIONED
ASSOCIATED WITH MODEL VERSION

135. DATABASE INTEGRATION

The geometry layer uses:

`geometry.geometry_references`

as the primary relational persistence reference.

Additional future tables may include:

`geometry.geometry_builds`

geometry.geometry_dependencies
geometry.geometry_artifacts
geometry.geometry_cache
geometry.spatial_indexes

These are derived computational structures.

136. PROPOSED GEOMETRY BUILD TABLE

Future migration:

geometry.geometry_builds

Fields:

id UUID PRIMARY KEY
project_id UUID NOT NULL
model_version_id UUID NOT NULL
engine_version VARCHAR(64) NOT NULL
input_hash VARCHAR(128) NOT NULL
output_hash VARCHAR(128)
build_mode VARCHAR(32) NOT NULL
status VARCHAR(32) NOT NULL
objects_processed INTEGER
objects_failed INTEGER
started_at TIMESTAMPTZ
completed_at TIMESTAMPTZ
diagnostics JSONB
created_at TIMESTAMPTZ NOT NULL

137. PROPOSED GEOMETRY DEPENDENCY TABLE

geometry.geometry_dependencies

Fields:

id UUID PRIMARY KEY
project_id UUID NOT NULL
source_geometry_id UUID NOT NULL
target_geometry_id UUID NOT NULL
dependency_type VARCHAR(64) NOT NULL
created_at TIMESTAMPTZ NOT NULL

138. GEOMETRY ARTIFACT TABLE

geometry.geometry_artifacts

Fields:

```
id UUID PRIMARY KEY
geometry_reference_id UUID NOT NULL
format VARCHAR(32) NOT NULL
storage_reference TEXT NOT NULL
content_hash VARCHAR(128) NOT NULL
size_bytes BIGINT
created_at TIMESTAMPTZ NOT NULL
```

139. GEOMETRY API

Internal API namespace:

/internal/geometry

Initial operations:

```
POST /build
GET /builds/{job_id}
GET /objects/{object_id}
POST /query
POST /validate
POST /preview
```

These are internal service interfaces and are not initially exposed directly to external customers.

140. GEOMETRY OBJECT REQUEST

```
{
  "object_id": "uuid",
  "model_version_id": "uuid",
  "detail_level": "DOCUMENTATION"
}
```

Response:

```
{
  "data": {
    "object_id": "uuid",
    "geometry_reference_id": "uuid",
    "geometry_type": "SOLID",
    "geometry_version": 3,
    "geometry_hash": "hash",
    "bounding_box": {}
  }
}
```

```
}  
}
```

141. GEOMETRY QUERY

```
{  
  "query_type": "INTERSECTING_BOX",  
  "box": {  
    "min": {},  
    "max": {}  
  },  
  "model_version_id": "uuid"  
}
```

142. GEOMETRY VALIDATION REQUEST

```
{  
  "project_id": "uuid",  
  "model_version_id": "uuid",  
  "geometry_version": 17,  
  "checks": [  
    "TOPOLOGY",  
    "INTERSECTIONS",  
    "HOST_RELATIONSHIPS",  
    "BOUNDARIES",  
    "CONSTRAINTS"  
  ]  
}
```

143. GEOMETRY VALIDATION RESPONSE

```
{  
  "data": {  
    "status": "PASS",  
    "errors": 0,  
    "warnings": 2,  
    "results": []  
  }  
}
```

144. GEOMETRY EVENT MODEL

The geometry service emits events such as:

GEOMETRY_BUILD_STARTED
GEOMETRY_BUILD_COMPLETED
GEOMETRY_BUILD_FAILED
GEOMETRY_INVALIDATED
GEOMETRY_ACTIVATED
GEOMETRY_VALIDATION_FAILED

Events are recorded through the existing audit/event architecture.

145. EVENT PAYLOAD

Example:

```
{  
  "event_type": "GEOMETRY_BUILD_COMPLETED",  
  "project_id": "uuid",  
  "model_version_id": "uuid",  
  "geometry_version": 17,  
  "geometry_hash": "hash",  
  "objects_processed": 42  
}
```

146. SECURITY BOUNDARY

The geometry worker must not possess unrestricted application authority.

Its permissions should be limited to:

```
read model state  
read geometry inputs  
write geometry artifacts  
write geometry metadata  
emit geometry events
```

It must not directly modify:

```
billing  
identity  
permissions  
organization membership  
customer acceptance
```

147. FAILURE RECOVERY

If a geometry worker fails:

CURRENT MODEL

remains unchanged.

The failed job becomes:

FAILED

The system retains the diagnostics.

A retry may be initiated.

148. RETRY SAFETY

Retries must use the same:

model_version_id
input_hash

unless the model has changed.

If the model has changed:

old job → STALE
new job → CREATED

149. CONCURRENCY

Two commands attempting to modify the same current revision must not silently overwrite one another.

The command boundary shall use:

expected_revision

or equivalent optimistic concurrency control.

150. GEOMETRY CONCURRENCY

Geometry jobs may execute concurrently for different model versions.

Only the geometry corresponding to the current accepted model version may become active project geometry.

151. AUDITABILITY

Every geometry modification must be traceable to:

- user
- Aria command
- revision
- model version
- geometry build
- geometry engine version
- geometry hash

This establishes the complete computational chain.

152. RECONSTRUCTION

The system must eventually be capable of:

- Project ID
- +
- Revision ID
- +
- Model Version ID

producing:

- same parameters
- same dependencies
- same constraints
- same geometry
- same geometry hash

provided the same engine version and computational inputs are used.

153. GEOMETRY PROVENANCE CHAIN

The complete chain becomes:

```
USER REQUIREMENT
  ↓
ARIA INTERPRETATION
  ↓
DOMAIN COMMAND
  ↓
MODEL VERSION
  ↓
PARAMETERS
  ↓
CONSTRAINTS
  ↓
```

GEOMETRY BUILD
↓
GEOMETRY VERSION
↓
GEOMETRY HASH
↓
VIEW
↓
DRAWING
↓
DOCUMENT PACKAGE

This chain is fundamental to OneDraft.

154. FIRST GEOMETRY ENGINE IMPLEMENTATION

The first implementation should not attempt to solve every architectural geometry problem.

The MVP geometry kernel shall implement:

Point
Vector
Line
Polyline
Polygon
Transform
Bounding Box
Wall
Door Opening
Window Opening
Floor Slab
Space Boundary
Basic Boolean Operations
Basic Intersection
Basic Constraint Evaluation
Geometry Hashing
Geometry Versioning

155. FIRST PARAMETRIC BUILDING

The first automated building should contain:

Site
Building
Ground Level
Level 1
Rooms
Exterior Walls
Interior Walls

Doors
Windows
Floor Slabs
Roof

This provides sufficient complexity to exercise the entire geometry stack.

156. FIRST GEOMETRY REGRESSION PROJECT

The canonical development project shall contain:

Residential Building
2 Levels
4 Exterior Walls
Interior Partition System
6 Doors
8 Windows
2 Floor Slabs
1 Roof
4 Rooms
1 Stair

Exact dimensions shall be stored in test fixtures.

157. FIRST GEOMETRY INTEGRATION TEST

The test shall execute:

1. Create project
2. Create building
3. Create Ground Level
4. Create Level 1
5. Create exterior wall loop
6. Create interior partitions
7. Create rooms
8. Create doors
9. Create windows

10. Generate wall solids
 11. Generate openings
 12. Generate floor slabs
 13. Generate roof
 14. Calculate room areas
 15. Calculate building envelope
 16. Run geometry validation
 17. Generate plan geometry
 18. Generate section geometry
 19. Generate elevation geometry
 20. Hash geometry
 21. Store geometry version
 22. Modify one wall
 23. Incrementally regenerate affected geometry
 24. Verify unaffected geometry remains unchanged
 25. Generate Revision 2
 26. Verify drawing invalidation
 27. Rebuild affected documentation
-

158. FIRST PARAMETRIC MODIFICATION TEST

Initial command:

MOVE_ELEMENT

Target:

rear_wall

Operation:

300 mm NORTH

Expected:

wall coordinates change
dependent openings move
affected room boundary changes
room area recalculates
affected section changes
affected elevation changes
affected drawings become stale
new geometry hash generated
new model version created

159. SECOND PARAMETRIC MODIFICATION TEST

Command:

RESIZE_ELEMENT

Example:

window width:
1200 mm → 1500 mm

Expected:

window geometry changes
opening changes
host wall geometry changes
affected elevation changes
affected section changes
affected schedule changes
validation is invalidated

160. THIRD PARAMETRIC MODIFICATION TEST

Command:

CHANGE_ASSEMBLY

Example:

wall assembly:
EXT-WALL-01 → EXT-WALL-02

Expected:

wall thickness changes

wall geometry changes
room boundary may change
building envelope may change
quantities change
affected documentation invalidated

161. FOURTH PARAMETRIC MODIFICATION TEST

Command:

CHANGE_LEVEL_ELEVATION

Expected:

dependent elements move vertically
stairs recalculate
roof relationships recalculate
sections update
elevations update
document package becomes stale

162. GEOMETRY ENGINE SUCCESS CRITERIA

The geometry engine is successful when it can:

Generate deterministic building geometry from semantic model data.

Maintain parametric relationships.

Evaluate constraints.

Generate valid architectural solids.

Generate openings.

Calculate spaces.

Detect geometric conflicts.

Regenerate incrementally.

Version geometry.

Hash geometry.

Reject stale jobs.

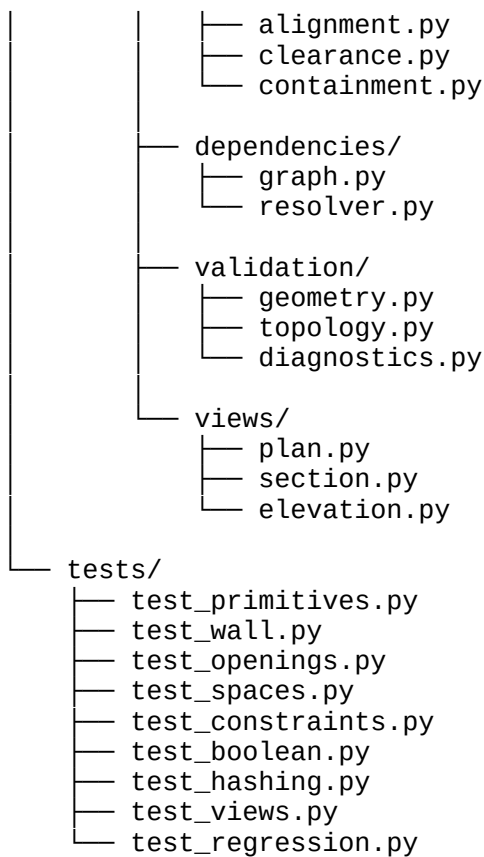
Support plan, section and elevation generation.

Provide reproducible geometry for documentation.

163. IMPLEMENTATION PACKAGE

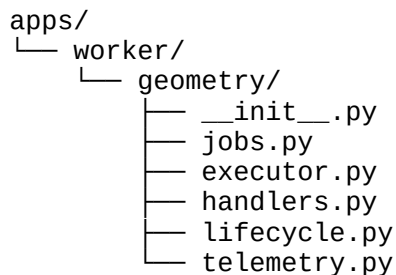
The next executable implementation package shall contain:

```
packages/
├── geometry/
│   ├── README.md
│   └── pyproject.toml
├── src/
│   ├── onedraft_geometry/
│   │   ├── __init__.py
│   │   ├── primitives.py
│   │   ├── vectors.py
│   │   ├── transforms.py
│   │   ├── curves.py
│   │   ├── polygons.py
│   │   ├── surfaces.py
│   │   ├── solids.py
│   │   ├── topology.py
│   │   ├── bounding_box.py
│   │   ├── tolerance.py
│   │   └── hashing.py
│   ├── parameters/
│   │   ├── __init__.py
│   │   ├── wall.py
│   │   ├── door.py
│   │   ├── window.py
│   │   ├── slab.py
│   │   ├── roof.py
│   │   └── space.py
│   ├── generators/
│   │   ├── wall.py
│   │   ├── opening.py
│   │   ├── slab.py
│   │   ├── roof.py
│   │   ├── stair.py
│   │   └── space.py
│   ├── operations/
│   │   ├── boolean.py
│   │   ├── intersect.py
│   │   ├── offset.py
│   │   ├── split.py
│   │   └── transform.py
│   └── constraints/
│       ├── evaluator.py
│       └── dimensional.py
```



164. GEOMETRY WORKER PACKAGE

The worker shall be added beneath:



The worker consumes geometry jobs from the application queue.

165. DOMAIN SERVICE INTEGRATION

The existing:

project_model_service.py

becomes the command boundary.

It invokes:

GeometryBuildService

after model mutation.

The relationship becomes:

```
project_model_service
  ↓
geometry_build_service
  ↓
geometry kernel
```

166. API INTEGRATION

The external API remains:

POST /api/v1/projects/{project_id}/aria/requests

Aria commands eventually invoke:

Project Model Service

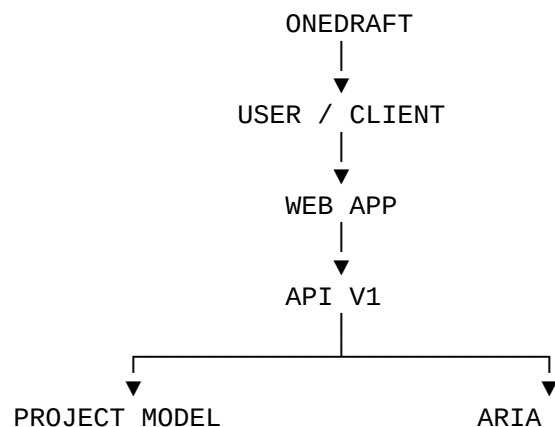
which invokes:

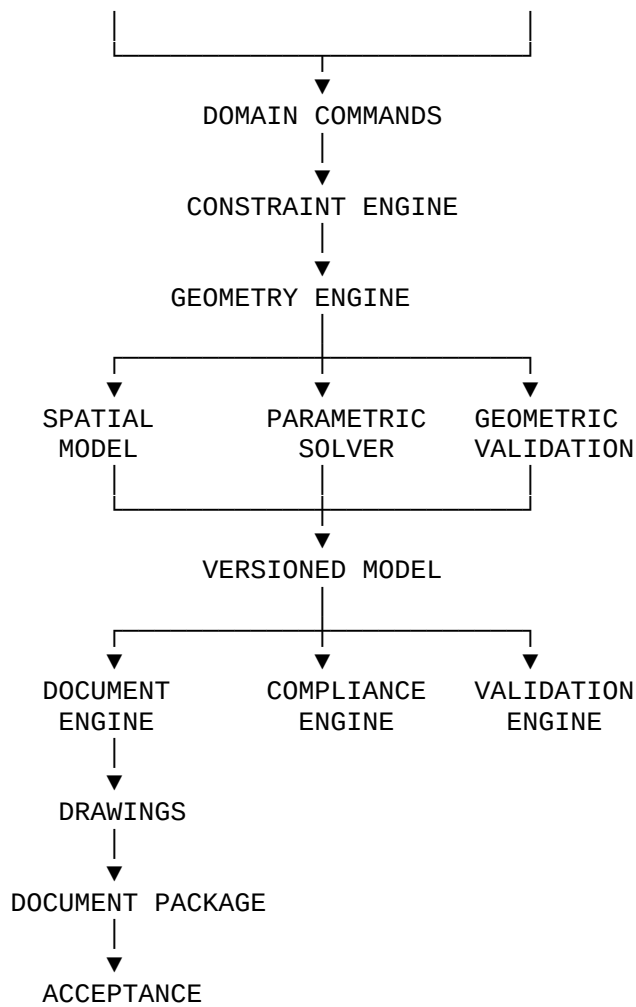
Geometry Service

External consumers do not need to understand the internal geometry kernel.

167. STACK POSITION

The OneDraft stack now becomes:





168. CURRENT ONEDRAFT ENGINEERING STATE

The architecture now contains:

Product definition

System architecture

Technology architecture

Domain model

Persistence model

API contract

Versioning architecture

AI command boundary
Regulatory model
Validation model
Documentation model
Redline workflow
Acceptance workflow
Geometry architecture
Parametric object model
Constraint architecture
Dependency graph
Geometry versioning
Geometry provenance
Incremental regeneration
Spatial computation
Plan generation
Section generation
Elevation generation
Geometry testing architecture

169. NEXT STACK LAYER

The next major layer after Geometry & Parametric Model Engine is:

COMPLIANCE & REGULATORY ENGINE

Its purpose is to transform jurisdiction-specific regulatory sources into a versioned computational ruleset capable of evaluating the OneDraft Project Model.

The resulting stack becomes:

```
PROJECT MODEL
  ↓
PARAMETRIC GEOMETRY
  ↓
COMPLIANCE / REGULATORY ENGINE
  ↓
VALIDATION ENGINE
```

↓
DOCUMENTATION ENGINE

The regulatory engine must therefore consume actual project geometry and semantic data rather than operate as a detached text-based chatbot.

170. FINAL ARCHITECTURAL DECISION

OneDraft Geometry shall not be implemented as:

IMAGE GENERATION
+
2D DRAWING MANIPULATION
+
STATIC CAD FILES

It shall be implemented as:

PARAMETRIC SEMANTIC OBJECTS
+
CONSTRAINTS
+
DEPENDENCY GRAPH
+
DETERMINISTIC GEOMETRY
+
VERSIONED STATE
+
SPATIAL COMPUTATION
+
DOCUMENTATION VIEWS

The geometry engine is therefore the computational bridge between the **OneDraft Project Model** and the **architectural drawing/documentation system**.

The drawing remains a derived representation.

The project model remains authoritative.

171. SPECIFICATION STATUS

ONEDRAFT GEOMETRY & PARAMETRIC MODEL ENGINE v0.1

STATUS: ESTABLISHED

The persistence/API layer is established.

The Geometry & Parametric Model Engine is now established as the next executable stack layer.

The next implementation artifacts are:

packages/geometry/

containing the geometry kernel, parametric generators, operations, constraints, dependency resolver, validation system and view generators;

apps/worker/geometry/

containing the asynchronous geometry execution worker;

and the integration of:

project_model_service.py

↓

geometry_build_service

↓

geometry kernel

The next major platform layer is:

ONEDRAFT – COMPLIANCE & REGULATORY ENGINE

which will connect jurisdiction-specific code sources and rules to the computational Project Model, geometry, constraints and validation pipeline.

172. STACK CONTINUITY

The OneDraft implementation sequence is therefore:

01 PROJECT DEFINITION

↓

02 SYSTEM ARCHITECTURE

↓

03 TECHNOLOGY STACK

↓

04 DOMAIN MODEL

↓

05 DATABASE / PERSISTENCE

↓

06 OPENAPI / MACHINE CONTRACT

↓

07 PROJECT MODEL SERVICE

↓

08 GEOMETRY & PARAMETRIC MODEL ENGINE

↓

09 COMPLIANCE & REGULATORY ENGINE

↓

10 VALIDATION ENGINE

↓

11 ARIA ORCHESTRATION ENGINE

↓

12 DOCUMENTATION / CAD ENGINE

↓

13 REDLINE / REVIEW ENGINE

↓

14 DOCUMENT PACKAGE ENGINE
↓
15 CUSTOMER ACCEPTANCE
↓
16 BILLING / SUBSCRIPTION
↓
17 OBSERVABILITY / SECURITY
↓
18 PRODUCTION DEPLOYMENT

This preserves the central OneDraft principle:

REQUIREMENTS
↓
COMPUTATIONAL PROJECT MODEL
↓
PARAMETRIC GEOMETRY
↓
REGULATORY VERIFICATION
↓
VALIDATED MODEL
↓
ARCHITECTURAL DOCUMENTATION
↓
CUSTOMER REVIEW
↓
FINAL DOCUMENT PACKAGE

.